Архитектурные решения

ADR-01. Локальная Qwen и vLLM

Контекст: нужен русский язык, автономный inference и consumer GPU с 12 GiB VRAM. Альтернативы: FP16 7B требует больше памяти; GGUF/llama.cpp удобен для CPU/offload; крупные модели требуют дополнительных GPU. Решение: Qwen2.5-7B-Instruct-AWQ и vLLM с AWQ Marlin, prefix caching и контекстом 4096. Следствие: компактный локальный serving и batching; квантование и ограничение контекста требуют оценки качества на выбранном корпусе. Используется готовая модель Apache-2.0, обучение не выполняется.

ADR-02. GraphRAG с provenance

Контекст: нужные сведения могут находиться в нескольких документах. Альтернативы: только векторный поиск не раскрывает связи; произвольное LLM-извлечение triples усложняет воспроизводимость и права. Решение: Document, Chunk, Entity, CO_MENTION и source chunk для каждого ребра, обход до двух шагов. SQLite хранит канонический граф, Neo4j служит производным представлением. Следствие: объяснимые связи и проверяемый RBAC; совместное упоминание не доказывает физическую зависимость, regex сущностей ограничен техническими идентификаторами.

ADR-03. Qdrant и автономные embeddings

Контекст: требуется self-hosted vector DB без runtime download. Альтернативы: Milvus сложнее в эксплуатации, managed DB нарушает границы контура. Решение: Qdrant с ACL payload и канонической проверкой SQL. Небольшой корпус использует lexical hash vectors; семантическая модель подключается из локального каталога. Следствие: воспроизводимость и изоляция индекса; смена dimension/model требует перестроения, stale payload не заменяет проверку актуальных прав.

ADR-04. Stateful LangGraph

Контекст: поиск требует памяти и условной повторной попытки. Альтернативы: линейная цепочка не поддерживает этот сценарий, сеть нескольких агентов избыточна для read-only поиска. Решение: StateGraph с типизированным state, conditional edges, двумя попытками и памятью, ограниченной владельцем сессии. Следствие: видимые переходы и ограниченное выполнение; для нескольких реплик нужна общая БД и checkpointer.

ADR-05. Проверяемые цитаты и атомарная авторизация

Контекст: секретный документ нельзя раскрыть через shared entity, историю или сгенерированный ответ. Альтернативы: фильтрация после LLM раскрывает данные модели; свободный ответ сложно автоматически связать с источниками. Решение: ACL до retrieval/traversal, выбор коротких source IDs моделью, восстановление полных исходных фрагментов с offsets, финальная проверка в BEGIN IMMEDIATE. Текст, версия, tenant, grants и сессия согласуются в одном snapshot. Следствие: меньше свободы формулировок, зато проверяемый provenance. SSE передаёт один проверенный результат; уже переданные байты не отзываются.

ADR-06. Локальная наблюдаемость и Compose

Контекст: нужны traces, метрики и автономное развёртывание. Альтернативы: облачная телеметрия выводит данные за контур, Kubernetes усложняет single-host MVP. Решение: OpenTelemetry, Jaeger, Prometheus/Grafana и Docker Compose с internal network. Следствие: наблюдаемость без содержимого документов; высокая доступность, Vault, SSO и mTLS предусмотрены целевой схемой и требуют отдельного развёртывания.

ADR-07. Точный бюджет контекста

Контекст: приблизительный подсчёт токенов может обрезать отрицание или условие в источнике. Альтернативы: обрезка по символам проще, но меняет смысл; большой контекст увеличивает VRAM и latency. Решение: фактический tokenizer/chat template vLLM, резерв ответа 700 и запас 64, упаковка полных единиц. Некорректный выбор модели или переполнение вызывают явный extractive fallback с той же валидацией. Следствие: дополнительный tokenizer RPC, сохранение целостности цитат и явный режим ответа.

ADR-08. Версии производных индексов

Контекст: очистка активных коллекций перед перестроением мешает запросам и откату. Альтернативы: распределённая транзакция между Qdrant и Neo4j сложна для MVP. Решение: новая collection/partition, проверка hashes, ACL, coverage и canonical epoch, затем публикация одного SQL pointer. Запрос считывает активную версию один раз. Следствие: дополнительные storage и время build; rollback разрешён только при совместимости канонических данных, удаление версий учитывает active pointer и grace period.